



BLM 4522 – AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ FİNAL PROJE RAPORU

GitHub: <https://github.com/semihcngz/AgTabaliParalelDagitimSistemleri>

Semih Cengiz

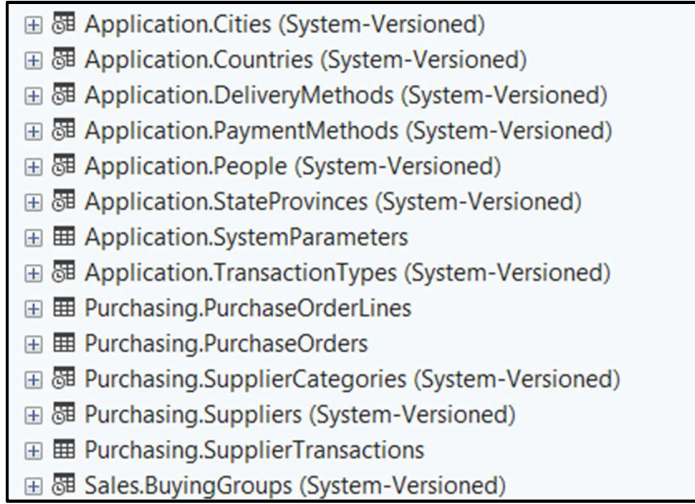
20290233

1. VERİTABANI GÜVENLİĞİ VE ERİŞİM KONTROLÜ

Bu çalışmada MSSQL üzerinde veritabanı güvenliği ve erişim kontrolü uygulamaları anlatılmıştır. Bu ve bundan sonraki projelerde örnek veritabanı olarak **WideWorldImporters** kullanılacaktır. Çalışmanın temel amacı, bir veritabanı sisteminde kullanıcı erişimlerinin nasıl yönetildiğini, hassas verilerin nasıl korunabileceğini, SQL Injection gibi saldırılara karşı veritabanını nasıl korumamız gerektiğini ve kullanıcı hareketlerini nasıl kayıt altına alınabileceğini göstermektir.

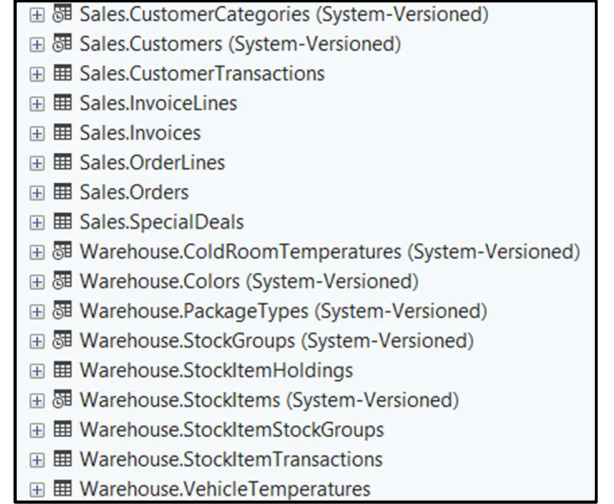
1.1. Örnek Veritabanı: WideWorldImporters

Bu veritabanında gerçek hayattaki ticari işlemleri temsil eden tablolar bulunmaktadır. Müşteriler, siparişler, tedarikçiler, satış işlemleri ve stok gibi farklı alanlara ait tablolar örnek verilebilir.



Application.Cities (System-Versioned)
Application.Countries (System-Versioned)
Application.DeliveryMethods (System-Versioned)
Application.PaymentMethods (System-Versioned)
Application.People (System-Versioned)
Application.StateProvinces (System-Versioned)
Application.SystemParameters
Application.TransactionTypes (System-Versioned)
Purchasing.PurchaseOrderLines
Purchasing.PurchaseOrders
Purchasing.SupplierCategories (System-Versioned)
Purchasing.Suppliers (System-Versioned)
Purchasing.SupplierTransactions
Sales.BuyingGroups (System-Versioned)

Şekil 2 Veritabanı İçindeki Tablolar



Sales.CustomerCategories (System-Versioned)
Sales.Customers (System-Versioned)
Sales.CustomerTransactions
Sales.InvoiceLines
Sales.Invoices
Sales.OrderLines
Sales.Orders
Sales.SpecialDeals
Warehouse.ColdRoomTemperatures (System-Versioned)
Warehouse.Colors (System-Versioned)
Warehouse.PackageTypes (System-Versioned)
Warehouse.StockGroups (System-Versioned)
Warehouse.StockItemHoldings
Warehouse.StockItems (System-Versioned)
Warehouse.StockItemStockGroups
Warehouse.StockItemTransactions
Warehouse.VehicleTemperatures

Şekil 1 Veritabanı İçindeki Tablolar Devamı

1.2. Erişim Yönetimi

Bir veritabanında her kullanıcının tüm yetkilere sahip olması güvenlik ve gizlilik açısından doğru değildir. Bu nedenle kullanıcılara yalnızca ihtiyaç duydukları yetkiler verilmelidir. SQL Server'da erişim yönetimi temel olarak dört yapı üzerinden gerçekleştirilir.

- Login -> SQL Server sunucusuna bağlanmak için kullanılan kimliktir.
- User -> Belirli bir veritabanı içerisindeki kullanıcıdır.
- Role -> Birden fazla kullanıcıya ortak yetki vermek için kullanılan gruptur.
- Permission -> SELECT, INSERT, UPDATE, DELETE gibi işlem izinleridir.

1.2.1. Login, User ve Role Oluşturma

Projede farklı erişim senaryolarını göstermek için 3 farklı SQL login oluşturdum. Bu kullanıcılar sadece okuma yetkisine sahip, okuma ve güncelleme yetkisine sahip ve belirli tabloya erişimi engellenmiş kullanıcı senaryolarını temsil etmektedir.

```
CREATE LOGIN wwi_sales_readonly  
WITH PASSWORD = 'readonlyPassword!123'  
  
CREATE LOGIN wwi_sales_writer  
WITH PASSWORD = 'writerPassword!123!'  
  
CREATE LOGIN wwi_denied_user  
WITH PASSWORD = 'userPassword!123!'
```

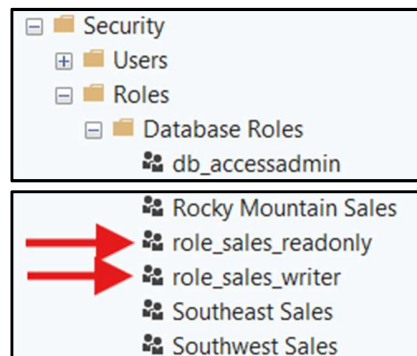
Şekil 3 Login Oluşturma

Daha sonra oluşturduğum login hesaplarını WideWorldImporters veritabanı içinde database user hesapları ile eşleştirdim. Bu sayede kullanıcının sunucuya bağlanması ve ilgili veritabanına yetkilendirilmesi sağlanmış oldu.

```
CREATE USER wwi_sales_readonly FOR LOGIN wwi_sales_readonly;  
CREATE USER wwi_sales_writer FOR LOGIN wwi_sales_writer;  
CREATE USER wwi_denied_user FOR LOGIN wwi_denied_user;
```

Şekil 4 Oluşturulan Kullanıcılar

Yetkilerin doğrudan kullanıcılara verilmesi yerine roller üzerinden verilmesi daha düzenli bir yönetim sağlar. Bu nedenle projede role_sales_readonly ve role_sales_writer adında 2 tane rol oluşturdum.



Şekil 5 Veritabanı Roller

1.2.2. Grant ve Deny Kullanımı

Yetki yönetimi için GRANT, DENY ve REVOKE komutları kullanılır. GRANT komutu izin verir, DENY komutu belirli bir yetkisi kesin olarak yasaklamak için kullanılır ve REVOKE komutu ise daha önce verilen veya engellenen izni kaldırılır.

```
GRANT SELECT ON SCHEMA::Sales TO role_sales_readonly;  
GRANT SELECT ON SCHEMA::Sales TO role_sales_writer;  
GRANT UPDATE ON SCHEMA::Sales TO role_sales_writer;
```

Şekil 6 Rol Yetkisi Verme

Bu işlem sonucunda okuma rolündeki kullanıcı Sales şemasındaki tabloları sadece okuyabilirken, yazma rolündeki kullanıcı hem okuyup hem de yazabilmektedir/güncelleyebilmektedir.

```
EXECUTE AS USER = 'wwi_sales_readonly';  
  
SELECT TOP 5 *  
FROM Sales.Orders;  
  
REVERT;
```

Results		Messages							
	OrderID	CustomerID	SalespersonPersonID	PickedByPersonID	ContactPersonID	BackorderOrderID	OrderDate	ExpectedDeliveryDate	CustomerPurchaseOrderN
1	1	832	2	NULL	3032	45	2013-01-01	2013-01-02	12126
2	2	803	8	NULL	3003	46	2013-01-01	2013-01-02	15342
3	3	105	7	NULL	1209	47	2013-01-01	2013-01-02	12211
4	4	57	16	3	1113	NULL	2013-01-01	2013-01-02	17129
5	5	905	3	NULL	3105	48	2013-01-01	2013-01-02	10369

Şekil 7 Select Başarı İle Gerçekleşti

```
EXECUTE AS USER = 'wwi_sales_readonly';

UPDATE Sales.Orders
SET OrderDate = '2026-05-28'
WHERE OrderID = 1;

REVERT;
```

```
Msg 229, Level 14, State 5, Line 3
The UPDATE permission was denied on the object 'Orders', database 'WideWorldImporters', schema 'Sales'.

Completion time: 2026-05-28T16:40:29.1466760+03:00
```

Şekil 8 Update Hatası

Şekil 7 ve 8’de görüldüğü gibi readonly kullanıcısı SELECT komutunu çalıştırabilirken UPDATE komutunu çalıştırmak istediğinde hata vermiştir.

```
GRANT SELECT ON SCHEMA::Sales TO wwi_denied_user;
DENY SELECT ON OBJECT::Sales.Customers TO wwi_denied_user;
```

Şekil 9 Deny Komutu

Ayrıca DENY komutu ile genel bir yetki verilmiş kullanıcının, özel olarak erişmesini istemediği tabloya erişimini engelleyebiliriz.

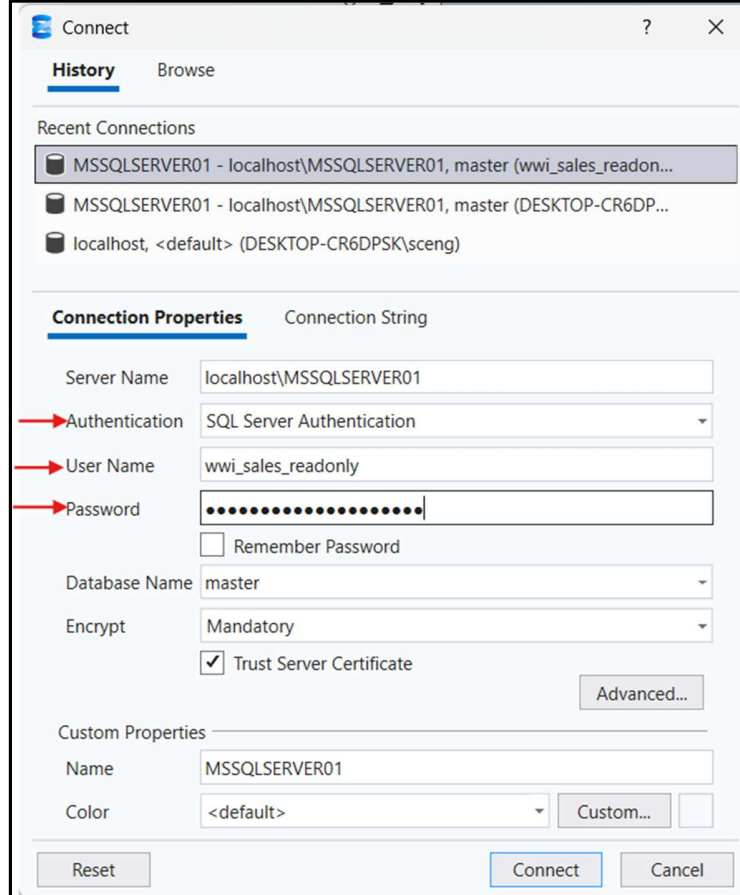
```
REVOKE UPDATE ON Sales.Orders FROM wwi_sales_readonly;
```

Şekil 10 Revoke Komutu

Şekil 10’da gösterilen komut çalıştırdıktan sonra kullanıcı tabloyu hala okuyabilir fakat güncelleyemez.

1.3. SQL Server Authentication ve Windows Authentication gösterimi

Windows Authentication yönteminde kullanıcı Windows hesabı üzerinden doğrulanır. SQL Server Authentication yönteminde ise kullanıcı adı ve parola SQL Server içerisinde tanımlanır.



Şekil 11 SQL Server Authentication

Şekil 11’de gösterildiği gibi daha önce oluşturduğum readonly kullanıcısı ile giriş yapıyorum. Bu kullanıcı sadece kendisine atanan rollere uygun işlemler yapabilecek.

1.4. Veri Şifreleme

Veritabanlarında hassas verilerin korunması için çeşitli şifreleme yöntemleri kullanılabilir. Örnek olarak Transparent Data Encryption, yani TDE anlatacağım. TDE, veritabanı dosyalarını disk üzerinde şifrelemek için kullanılan bir özelliktir. Bu sayede veritabanı dosyalarının veya yedeklerin istenmeyen kişilerin eline geçmesi halinde verilerin korunmasına, erişilememesini sağlar.

TDE uygulamasında önce master key oluşturulur, ardından sertifika oluşturulur. Daha sonra veritabanı encryption key tanımlanır ve ilgili veritabanı için encryption aktif edilir.

```
USE master;
GO
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'MasterKeyPassword!123';
GO

CREATE CERTIFICATE WWI_TDE_Certificate
WITH SUBJECT = 'TDE Certificate for WideWorldImporters';
GO

USE WideWorldImporters;
GO
CREATE DATABASE ENCRYPTION KEY
WITH ALGORITHM = AES_256
ENCRYPTION BY SERVER CERTIFICATE WWI_TDE_Certificate;
GO

ALTER DATABASE WideWorldImporters
SET ENCRYPTION ON;
GO
```

Şekil 12 Sertifika Oluşturma Aşamaları

	DatabaseName	encryption_state	percent_complete	key_algorithm	key_length
1	WideWorldImporters	3	0	AES	256

Şekil 13 Sertifika Kontrol

Oluşturduğumuz sertifikayı **sys.dm_database_encryption_keys** üzerinden kontrol ettiğimizde şekil 13'teki tabloya ulaşabiliyoruz. Encryption_state aşaması şifrelemenin hangi durumda olduğunu bize gösteriyor. 3, şifrelenmiş anlamına gelmektedir.

1.4.1. Sertifika Yedekleme

Bu aşama çok önemli çünkü eğer sertifikayı kaybedersek daha sonra yapılacak backup restore işlemlerinde veritabanını geri getiremeyiz.

```

USE master;
GO

BACKUP CERTIFICATE WWI_TDE_Certificate
TO FILE = 'C:\SQLBackups\TDE\WWI_TDE_Certificate.cer'
WITH PRIVATE KEY (
    FILE = 'C:\SQLBackups\TDE\WWI_TDE_PrivateKey.pvk',
    ENCRYPTION BY PASSWORD = 'PrivateKeyPassword!123'
);
GO

```

Şekil 14 Sertifika Yedekleme

Name	Date modified	Type	Size
 WWI_TDE_Certificate.cer	28.05.2026 19:38	Security Certificate	1 KB
 WWI_TDE_PrivateKey.pvk	28.05.2026 19:38	PVK File	2 KB

Şekil 15 Sertifika ve Key Klasörde Gösterme

TDE sertifikası ve private key dosyası yedeklenerek, veritabanının başka bir sunucuya yedeklenmesi halinde gerekli olan materyaller korunmuştur.

1.5. SQL Injection Testleri

SQL Injection kullanıcıdan alınan girdilerin doğrudan SQL sorgularına eklenmesi sonucunda ortaya çıkan bir güvenlik sorunudur. Bu açık nedeniyle saldırganlar verilere yetkisiz erişebilir, sorgunun mantığını değiştirebilir veya veritabanına zarar verebilir. Bu aşamaları göstermek için öncelikle test tablosu oluşturuyorum.

```

CREATE TABLE dbo.LoginDemo (
    UserID INT IDENTITY(1,1) PRIMARY KEY,
    Username NVARCHAR(50) NOT NULL,
    PasswordText NVARCHAR(50) NOT NULL,
    RoleName NVARCHAR(50) NOT NULL
);

```

Şekil 16 SQL Injection İçin Test Ortamı

Test edebilmek için bu veritabanına kullanıcı şifrelerini düz metin olarak ekliyorum, normalde parolarlar hash'lenerek tutulur fakat burada test ortamı oluşturuyoruz

1.5.1. Güvensiz Sorgu Örneği

İlk olarak kullanıcı girdisini doğrudan SQL sorgusuna ekleyen güvensiz bir stored procedure oluşturdum. Bu yöntem, kullanıcı girdisi SQL komutunun bir parçası haline geldiği için güvenli değildir.

```
CREATE OR ALTER PROCEDURE dbo.UnsafeLoginSearch
    @Username NVARCHAR(100),
    @Password NVARCHAR(100)
AS
BEGIN
    DECLARE @sql NVARCHAR(MAX);

    SET @sql = N'
        SELECT UserID, Username, RoleName
        FROM dbo.LoginDemo
        WHERE Username = ''' + @Username + '''
        AND PasswordText = ''' + @Password + '''';

    PRINT @sql;

    EXEC(@sql);
END;
GO
```

Şekil 17 Güvensiz Sorgu Örneği

Burdaki sorun dışarıdan gelen kullanıcı girdisinin hiçbir filtrelemeden geçmeden direkt string birleştirme yöntemiyle doğrudan çalışacak şekilde SQL komutunun içine metin olarak gömülmesidir.

```
EXEC dbo.UnsafeLoginSearch  
  @Username = 'admin' OR 1=1 --',  
  @Password = 'rastgelesfreee';
```

Şekil 18 Saldırganın Yazdığı Sorgu

Buradaki fazla tırnak ve yorum satırı (--) yüzünden SQL Server stringleri birleştirdiğimizden dolayı durumu yanlış anlayarak şifre yanlış olsa bile kabul edecektir.

```
SELECT UserID, Username, RoleName  
FROM dbo.LoginDemo  
WHERE Username = 'admin' OR 1=1 --' AND PasswordText = 'rastgele'
```

Şekil 19 SQL Server'ın Anladığı Komut

Şekil 19'da da görüldüğü gibi yorum satırı sonrası önemini kaybediyor ve 1=1 TRUE olduğundan sorgu doğru kabul ediliyor. Bunun sonucunda da admin hesabının verileri başarıya geri döndürülür.

1.5.2. Güvenli Stored Procedure Oluşturma

SQL injection riskini azaltmak için parametrelili sorgular kullanılmalıdır. Parametrelili sorgularda kullanıcı girdisi SQL komutu olarak değil, veri olarak değerlendirilir.

```
CREATE OR ALTER PROCEDURE dbo.SafeLoginSearch  
  @Username NVARCHAR(100),  
  @Password NVARCHAR(100)  
AS  
BEGIN  
  SELECT UserID, Username, RoleName  
  FROM dbo.LoginDemo  
  WHERE Username = @Username  
        AND PasswordText = @Password;  
END;  
GO
```

Şekil 120 Güvenli Procedure

1.6. Audit Logları

Audit logları, veritabanı üzerinde yapılan işlemlerin izlenmesini sağlar. Bu sayede hangi kullanıcının, hangi zamanda, hangi işlem yaptığı kayıt altına alınabilir. Örneğin kim hangi tabloya ne zaman erişti, kim başarısız işlem yaptı, kim SELECT UPDATE denedi gibi. Güvenlik açısından audit logları önemlidir çünkü şüpheli işlemlerin tespit edilmesine ve sonradan incelenmesine yardımcı olur. Bunun için server audit oluşturarak başlayalım.

```
USE master;
GO

CREATE SERVER AUDIT WWI_Security_Audit
TO FILE (
    FILEPATH = 'C:\SQLAudit\',
    MAXSIZE = 50 MB,
    MAX_ROLLOVER_FILES = 10
)
WITH (
    QUEUE_DELAY = 1000,
    ON_FAILURE = CONTINUE
);
GO

ALTER SERVER AUDIT WWI_Security_Audit
WITH (STATE = ON);
GO
```

Şekil 21 Server Audit

Server seviyesinde audit oluşturduktan sonra veritabanı için için database audit specification oluşturdum. Bu yapı sayesinde istediğimiz işlemlerin kayıtları audit dosyalarına kaydedilmektedir.

```
CREATE DATABASE AUDIT SPECIFICATION WWI_Database_Audit_Spec
FOR SERVER AUDIT WWI_Security_Audit
ADD (SELECT ON SCHEMA::Sales BY public),
ADD (UPDATE ON SCHEMA::Sales BY public),
ADD (SELECT ON OBJECT::dbo.LoginDemo BY public),
ADD (EXECUTE ON OBJECT::dbo.SafeLoginSearch BY public)
WITH (STATE = ON);
GO
```

Şekil 22 Audit Specification

Yapılan işlem sayesinde şu işlemlerin log'ları kaydedilir:

- Sales şemasında SELECT
- Sales şemasında UPDATE
- LoginDemo tablosunda SELECT
- SafeLoginSearch procedure çalıştırma

	event_time	action_id	succeeded	server_principal_name	database_principal_name	database_name
1	2026-05-29 10:03:59.6794915	SL	1	DESKTOP-CR6DPSKİsceng	dbo	WideWorldImporters
2	2026-05-29 10:03:59.6784923	EX	1	DESKTOP-CR6DPSKİsceng	dbo	WideWorldImporters
3	2026-05-29 10:03:59.6774969	SL	1	DESKTOP-CR6DPSKİsceng	dbo	WideWorldImporters
4	2026-05-29 10:03:54.4865669	SL	0	wwi_sales_readonly	wwi_sales_readonly	WideWorldImporters
5	2026-05-29 10:01:59.4773636	SL	0	wwi_sales_readonly	wwi_sales_readonly	WideWorldImporters

schema_name	object_name	statement
dbo	LoginDemo	SELECT UserID, Username, RoleName FROM dbo....
dbo	SafeLoginSearch	EXEC dbo.SafeLoginSearch @Username = 'semih', ...
Sales	Customers	SELECT TOP 5 * FROM Sales.Customers
dbo	LoginDemo	SELECT * FROM dbo.LoginDemo
dbo	LoginDemo	SELECT * FROM dbo.LoginDemo

Şekil 23 Audit Logları